Programmieren 1

2. Tutorium · IOTools, Operatoren, Klasse Math, Typecasts, if/else & Arrays

Fokus heute: Aufgabenpool „Imperatives Programmieren“

- Keine Folien-Vorlesung – wir lösen heute Aufgaben direkt aus dem Pool!
- Ihr knobelt kurz am Platz – danach coden wir zusammen am Beamer
- Ziel: Lauffähiger Code, sauberes Verständnis & typische Klausurfallen

Unser Fahrplan für heute

Themenblöcke

1. Eingabe & IOTools: Prog1Tools einbinden & nutzen

2. Operatoren & Modulo: Ganzzahldivision vs. Komma

3. Klasse Math: sqrt, pow und PI für Berechnungen

4. Typecasts: Implizit & Explizit (Abschneiden)

5. Inkrement / Dekrement: Pre vs. Post (++a vs. a++)

6. Verzweigungen: if/else, switch & Klammern

7. Arrays: 1D-Felder, Indizes & Referenzen

Unser Fahrplan für heute

Themenblöcke

1. Eingabe & IOTools: Prog1Tools einbinden & nutzen

2. Operatoren & Modulo: Ganzzahldivision vs. Komma

3. Klasse Math: sqrt, pow und PI für Berechnungen

4. Typecasts: Implizit & Explizit (Abschneiden)

5. Inkrement / Dekrement: Pre vs. Post (++a vs. a++)

6. Verzweigungen: if/else, switch & Klammern

7. Arrays: 1D-Felder, Indizes & Referenzen

Aufgaben aus dem Pool

Aufgabe 3 (!) → Testeingabe mit IOTools

Aufgabe 1 → Modulo & Division

Aufgabe 5 → Raviolita (Klasse Math & Dosen)

Aufgabe 7 → Typecasts & Literale

Aufgabe 25 → Pre- und Post-Increment

Aufgabe 9 & 8 → Logische Fehler & Verschachtelung

Aufgabe 17 (!) → Array-Vergleich & Referenzen

Eingaben von der Konsole: IOTools

Wichtige Methoden

`readInteger()` → int einlesen

`readDouble()` → double einlesen

`readLine()` → ganze Zeile als String

`readString()` → nur 1 Wort als String

`readChar()` → einzelnes char

Prompt-Variante: Text direkt in die Klammern schreiben!

Eingaben von der Konsole: IOTools

Wichtige Methoden

`readInteger()` → int einlesen

`readDouble()` → double einlesen

`readLine()` → ganze Zeile als String

`readString()` → nur 1 Wort als String

`readChar()` → einzelnes char

Prompt-Variante: Text direkt in die Klammern schreiben!

IOTools Syntax

```
import Prog1Tools.IOTools;

// Liest direkt mit Aufforderung ein:
int alter = IOTools.readInteger("Alter: ");
String name = IOTools.readLine("Name: ");
```

Warum IOTools?

- Einfacher als Scanner / BufferedReader
- Fragt bei Falscheingabe automatisch erneut nach

Live-Setup: Prog1Tools in IntelliJ einbinden

1. Datei ablegen

- Prog1Tools.zip (aus ISIS) herunterladen
- Im Projekt einen Ordner 'lib' erstellen und Datei dort hineinlegen

Live-Setup: Prog1Tools in IntelliJ einbinden

1. Datei ablegen

- Prog1Tools.zip (aus ISIS) herunterladen
- Im Projekt einen Ordner 'lib' erstellen und Datei dort hineinlegen

2. Library registrieren

- File → Project Structure (Strg+Alt+Shift+S)
- Libraries → '+' → Java → Prog1Tools.zip auswählen → OK

Live-Setup: Prog1Tools in IntelliJ einbinden

1. Datei ablegen

- Prog1Tools.zip (aus ISIS) herunterladen
- Im Projekt einen Ordner 'lib' erstellen und Datei dort hineinlegen

2. Library registrieren

- File → Project Structure (Strg+Alt+Shift+S)
- Libraries → '+' → Java → Prog1Tools.zip auswählen → OK

3. Nutzen im Code

- Ganz oben in eure .java-Datei schreiben:
- import Prog1Tools.IOTools;
- Fertig – kein roter Fehler mehr!

Häufiger Fehler: Die .zip-Datei NICHT entpacken – IntelliJ kann direkt die .zip als Library einbinden!

Aufgabe 3 (!)

Testeingabe mit IOTools

Aufgabe: Aufgabe 3 (!)

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 1)

Ziel: Ein Programm schreiben, das nach Name und Alter (in Jahren) fragt, mit Namen begrüßt und das Alter in Tagen ausgibt.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. Name einlesen: readLine("Name: ") nutzen (Leerzeichen im Namen!)

2. Alter einlesen: readInteger("Alter: ") für ganze Jahre

3. Umrechnen: alter * 365 in einer int-Variable speichern

4. Ausgabe: Begrüßung und Tage mit '+' zu einem String verketten

💡 **Tipp:** Schaltjahre dürfen wir vernachlässigen (einfach * 365).

Rechnen in Java: Division, Modulo & Strings

Zahlenoperatoren

Ganzzahldivision: 10 / 3 ergibt 3 (Nachkommastellen fallen weg!)

Modulo (%): 10 % 3 ergibt 1 (der Divisionsrest)

Komma-Division: 10 / 3.0 ergibt 3.333… (ein double reicht)

Priorität: Klammern vor Punkt vor Strich

Auswertung: Bei gleicher Stufe strikt von links nach rechts

Rechnen in Java: Division, Modulo & Strings

Zahlenoperatoren

Ganzzahldivision: 10 / 3 ergibt 3 (Nachkommastellen fallen weg!)

Modulo (%): 10 % 3 ergibt 1 (der Divisionsrest)

Komma-Division: 10 / 3.0 ergibt 3.333... (ein double reicht)

Priorität: Klammern vor Punkt vor Strich

Auswertung: Bei gleicher Stufe strikt von links nach rechts

Der Plus-Operator (+)

Zwei Zahlen: 1 + 2 ergibt 3 (normale Addition)

Mit String: "Ergebnis: " + 1 + 2 ergibt "Ergebnis: 12"!

Warum? Von links nach rechts: Erst String + 1 = String, dann String + 2 = String

Lösung: "Ergebnis: " + (1 + 2) mit Klammern erzwingt 3

Aufgabe 1

Modulo-Operator & Typen

Aufgabe: Aufgabe 1

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 1)

Ziel: Die Rechenoperationen 10/3, 10/3.0 und 10%3 durchführen, Ergebnisse in passenden Variablen speichern und ausgeben.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. int-Division: int i = 10 / 3; → ergibt 3

2. double-Division: double d = 10 / 3.0; → ergibt 3.333…

3. Modulo-Rest: int j = 10 % 3; → ergibt 1

4. Bildschirmausgabe: Ergebnisse mit beschreibendem Text ausgeben

💡 **Tipp:** int x = 10 / 3.0; würde nicht kompilieren, weil double nicht ohne Cast in int passt!

Mathematische Berechnungen: Klasse Math

Wichtige Methoden & Konstanten

`Math.sqrt(d)` → Quadratwurzel √d als double

`Math.pow(a, b)` → Potenz a^b als double

`Math.abs(d)` → Betrag |d|

`Math.max(a, b)` → Größere der beiden Zahlen

`Math.random()` → Zufallszahl in [0.0 ; 1.0)

`Math.PI` → Kreiszahl π (ca. 3.14159…)

Mathematische Berechnungen: Klasse Math

Wichtige Methoden & Konstanten

`Math.sqrt(d)` → Quadratwurzel √d als double

`Math.pow(a, b)` → Potenz a^b als double

`Math.abs(d)` → Betrag |d|

`Math.max(a, b)` → Größere der beiden Zahlen

`Math.random()` → Zufallszahl in [0.0 ; 1.0)

`Math.PI` → Kreiszahl π (ca. 3.14159…)

Beispiel Dreieck

```
// Hypotenuse im rechtwinkligen Dreieck:
double a = 3.0, b = 4.0;
double c = Math.sqrt(Math.pow(a, 2) +
Math.pow(b, 2));
// oder kompakter:
double c2 = Math.sqrt(a * a + b * b);
System.out.println("c = " + c); // c = 5.0
```

Aufruf-Muster

Kein import nötig: Klasse Math gehört zum JDK-Kern (java.lang)

Rückgabetyp: Die meisten Math-Methoden liefern double!

Aufgabe 5

Konservendosen berechnen (Raviolita)

Aufgabe: Aufgabe 5

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 2)

Ziel: Aus Blechlänge u (Umfang) und Breite h (Höhe) alle Kenngrößen der Dose mit der Klasse Math berechnen und ausgeben.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. Eingabe: u und h mit IOTools.readDouble(...) einlesen

2. Diagonale: lDiag = Math.sqrt(u * u + h * h);

3. Boden: dBoden = u / Math.PI; fBoden = Math.PI * (dBoden/2)²;

4. Mantel & Gesamt: fMantel = u * h; fGesamt = 2 * fBoden + fMantel;

5. Volumen: v = fBoden * h; (alle Werte sauber ausgeben)

💡 **Tipp:** Umfang u = π · d → Durchmesser ist u / Math.PI. Alle Zwischenwerte als double speichern!

Typecasts: Typen konvertieren

Impliziter Cast (automatisch)

Regel: Von klein nach groß – kein Informationsverlust

Kette: byte → short → int → long → float → double

Zeichen: char → int (und größer) geht auch!

Beispiel: int a = 5; double d = a; // d = 5.0

Typecasts: Typen konvertieren

Impliziter Cast (automatisch)

Regel: Von klein nach groß – kein Informationsverlust

Kette: byte → short → int → long → float → double

Zeichen: char → int (und größer) geht auch!

Beispiel: int a = 5; double d = a; // d = 5.0

Expliziter Cast (manuell)

Syntax: (Zieltyp) wert

Wann nötig? Groß nach klein (Kompilierfehler ohne Cast!)

Achtung: Nachkommastellen werden abgeschnitten: (int) 3.99 ist 3!

Überlauf: (byte) 130 ergibt -126 (Bit-Überlauf)

Zeichen: (char) 65 ergibt 'A'

Aufgabe 7

Typecasts & Literale

Aufgabe: Aufgabe 7

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 4)

Ziel: Gegebenen Code auf Seite 4 analysieren und vorhersagen, was Java jeweils auf der Konsole ausgibt.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. Zeichen & Zahlen: (char) 65 → Buchstabe 'A' (ASCII-Code)

2. Byte-Überlauf: (byte) 10084 → 100 (nur die unteren 8 Bit bleiben)

3. Typfalle Division: double f = 3 / 2; → 1.0 (erst int teilen, dann zu double!)

4. Zahlensysteme: 0x12345 (Hexadezimal) und 012 (Oktal mit führender 0 = 10)

💡 **Tipp:** Klassiker: double f = 3 / 2; berechnet erst ganzzahlig 1 und macht DARAUS 1.0!

x++ vs. ++x: Wann wird erhöht?

Post-Inkrement: x++

Bedeutung: Erst alten Wert auswerten, DANACH x um 1 erhöhen

Beispiel: int x = 5; int a = x++;

Ergebnis: a erhält 5, danach wird x zu 6!

Genauso: x-- (Post-Dekrement)

x++ vs. ++x: Wann wird erhöht?

Post-Inkrement: x++

Bedeutung: Erst alten Wert auswerten, DANACH x um 1 erhöhen

Beispiel: int x = 5; int a = x++;

Ergebnis: a erhält 5, danach wird x zu 6!

Genauso: x-- (Post-Dekrement)

Pre-Inkrement: ++x

Bedeutung: Erst x um 1 erhöhen, DANACH neuen Wert auswerten

Beispiel: int x = 5; int a = ++x;

Ergebnis: x wird 6, a erhält 6!

Zusammengesetzt: a += b; ist gleich a = a + b;

Faustregel: Im echten Code ++ immer als eigene Zeile nutzen!

Aufgabe 25

Pre- und Post-Increment Knobeln

Aufgabe: Aufgabe 25

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 12)

Ziel: Für die Code-Sequenz auf Seite 12 den aktuellen Wert von a nach jedem Schritt auf Papier mitverfolgen.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. Start & Inkrement: a = 1; a++; → a ist 2

2. Die Zuweisungs-Falle: a = a++; → alter Wert 2 überschreibt das Inkrement!

3. Pre-Inkrement: a = ++a; → a wird 3

4. Rechenschritte: a += a; (ergibt 6) und a -= a + 1; (ergibt -1)

5. Finale: a = (a--) - (--a); → (-1) - (-3) = 2

💡 **Tipp:** Bei a = a++; wird der alte Wert (2) zwischengespeichert und überschreibt das anschließende Erhöhen wieder!

Verzweigungen: if / else & switch

if / else Regeln

Syntax: if (Bedingung) { ... } else { ... }

Verknüpfung: && (UND), || (ODER), ! (NICHT)

Goldene Regel: IMMER geschweifte Klammern { } setzen!

Gefahr ohne { }: Nur die nächste EINZELNE Anweisung gehört zum if – Einrückung wird von Java ignoriert!

Apple-Bug 2014: Schwere Sicherheitslücke wegen fehlender { }

Verzweigungen: if / else & switch

if / else Regeln

Syntax: if (Bedingung) { ... } else { ... }

Verknüpfung: && (UND), || (ODER), ! (NICHT)

Goldene Regel: IMMER geschweifte Klammern { } setzen!

Gefahr ohne { }: Nur die nächste EINZELNE Anweisung gehört zum if – Einrückung wird von Java ignoriert!

Apple-Bug 2014: Schwere Sicherheitslücke wegen fehlender { }

switch-case Regeln

Einsatz: Eine Variable auf konkrete feste Werte testen

Erlaubt (VL): int, char, String

break; Beendet den Block – OHNE break läuft Java in den nächsten case weiter (Fall-through)!

default: Greift, wenn kein case passte

Kompakter: Mehrere cases untereinander für gleiche Aktion

Aufgabe 9

Logische Fehler bei Verzweigungen

Aufgabe: Aufgabe 9

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 6)

Ziel: Vier Codebeispiele auf Seite 6 analysieren: Sie kompilieren fehlerfrei, verhalten sich aber falsch oder stürzen ab.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

Beispiel A (Dangling else): else bindet an das innere if – falsche Ausgabe bei x = 0

Beispiel B (Division durch 0): 1/x läuft IMMER, weil { } fehlen → Absturz bei x = 0

Beispiel C (Semikolon-Falle): if (x > 0); beendet das if sofort als Leerbefehl

Beispiel D (Variablentausch): x = y überschreibt x sofort → Hilfsvariable tmp nötig

💡 **Tipp:** In IntelliJ einfügen und Strg+Alt+L drücken – die automatische Einrückung deckt Fehler A sofort auf!

Aufgabe 8

Verschachtelte ifs entwirren

Aufgabe: Aufgabe 8

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 5)

Ziel: Einen stark verschachtelten if-else-Block auf Seite 5 analysieren: Was berechnet e? Eine Lösung mit nur 3 ifs finden.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. Analyse: Code mit Testwerten durchspielen (a=3, b=9, c=2, d=5)

2. Erkenntnis: e ist das Maximum aller 4 Variablen (hier e = 9.0)

3. Kürzere Version: Startwert: double e = a;

4. Nur 3 ifs nötig: if (b > e) e = b; if (c > e) e = c; if (d > e) e = d;

💡 **Tipp:** Strategie 'Bisheriges Maximum merken': e = a, dann nacheinander mit b, c, d vergleichen und bei Bedarf aktualisieren.

Arrays: Daten in einer Reihe

Array-Grundlagen

Was ist ein Array? Feste Gruppe von Variablen gleichen Typs

Indexierung: Startet IMMER bei 0 bis (Länge - 1)

Länge abfragen: arr.length (ohne Klammern!)

`Deklaration:` int[] arr;

`Initialisierung:` arr = new int[5];

`Zugriff:` arr[0] = 42; // Schreiben

int x = arr[0]; // Lesen

Arrays: Daten in einer Reihe

Array-Grundlagen

Was ist ein Array? Feste Gruppe von Variablen gleichen Typs

Indexierung: Startet IMMER bei 0 bis (Länge - 1)

Länge abfragen: arr.length (ohne Klammern!)

`Deklaration:` int[] arr;

`Initialisierung:` arr = new int[5];

`Zugriff:` arr[0] = 42; // Schreiben

int x = arr[0]; // Lesen

Referenztyp-Besonderheiten

Standardwerte: Neue int-Arrays sind automatisch voller Nullen (0)

Speicher: Die Variable arr speichert nur die ADRESSE, wo das Array im RAM liegt!

Kopie mit '=': int[] b = arr; kopiert nur die Adresse! Änderungen an b ändern auch arr!

Gefahr 1: arr[5] bei Länge 5 → ArrayIndexOutOfBoundsException

Gefahr 2: Zugriff vor 'new' → NullPointerException

Aufgabe 17 (!)

Vergleich von Arrays mit ==

Aufgabe: Aufgabe 17 (!)

Aufgabenblatt: 02-imperatives-programmieren.pdf (S. 9)

Ziel: Erklären, warum man zwei Arrays a und b auf Seite 9 nicht mittels a == b auf inhaltliche Gleichheit prüfen kann.

Modus: Zuerst kurz überlegen – dann live am Beamer coden!

Code-Fahrplan (Checkliste)

1. Code analysieren: int[] a = new int[2]; int[] b = new int[2]; System.out.println(a == b);

2. Überraschendes Ergebnis: Gibt false aus – obwohl beide Arrays mit Nullen gefüllt sind!

3. Warum false? == vergleicht bei Arrays nur die Speicheradresse im RAM, NICHT den Inhalt!

4. Echter Vergleich: Inhalte muss man Element für Element prüfen (Schleifen kommen nächste Woche!)

💡 **Tipp:** a und b wurden mit zwei separaten 'new'-Befehlen erzeugt → liegen an zwei verschiedenen Stellen im Speicher!

Geschafft für heute! 🎉

Das nehmt ihr mit

✔️ IOTools & Prog1Tools im Projekt eingerichtet

✔️ int/int schneidet ab – ein double erzwingt Komma

✔️ Klasse Math (sqrt, pow, PI) für Berechnungen

✔️ Immer { } bei if/else setzen!

✔️ Typecasts runden nie – sie schneiden ab

✔️ Arrays beginnen bei Index 0

✔️

Empfehlungen zum Weiterüben

Aufgabe 0: Pseudocode formulieren

Aufgabe 6: Rundungsfehler bei double

Aufgabe 11: Wochentagsberechnung (if/switch)

Aufgabe 21: Syntax- vs. Semantikfehler